

Проект «Эмулятор микрокалькулятора «Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

Черновик инструкции по FOCAL

Версия черновика: 06.07.2026

Этот документ описывает предлагаемую реализацию FOCAL для МК61s mini. Это не описание готовой функции, а рабочая спецификация перед разработкой.

FOCAL должен быть отдельным режимом со своим редактором. Главная идея - короткий строчный язык поверх возможностей МК-61: десятичные номера строк, компактные операторы, калькуляторный ввод выражений и доступ к вычислительным функциям МК-61.

Основные решения

- У FOCAL отдельный редактор и отдельное меню.
- Программа состоит из строк с номерами вида **1.10**, **1.20**, **2.10**.
- Номер строки хранится не как вещественное число, а как структурный номер с целой и дробной частью.
- Редактор показывает несколько строк программы: 2 строки на двухстрочном дисплее и 4 строки на графическом дисплее.
- Операторы отображаются короткими буквами и вводятся назначенными клавишами.
- Алиасы не используются: одно действие - одно имя.
- Синтаксис программы всегда латинский: операторы, функции и переменные записываются английскими именами даже в русском режиме интерфейса.
- В русском режиме локализуются меню, подсказки, сообщения и ошибки.
- FOCAL включен по умолчанию и должен отключаться условной компиляцией через **MK61_ENABLE_FOCAL**.
- **PRINT** используется вместо исторического FOCAL **TYPE**.
- **PRINT** поддерживает список элементов; **!** в списке означает перевод строки.
- **SET** всегда отображается явно как **S X=...**
- **FOR** повторяет один оператор, который записывается после **;**.
- **GOTO** используется для безусловного перехода, а **BRANCH** - для ветвления по знаку выражения.
- Программа может храниться в постоянном хранилище устройства, а не только в RAM.
- **EXIT** используется вместо **QUIT**.
- **RND()** вызывает тот же калькуляторный генератор, что операция МК-61 **СЧ**.
- Клавиши МК-61 для **x^2**, **1/x**, **sqrt**, **sin**, **cos** и других функций работают как редакторские макросы выражения.

Строки и номера

Строка начинается с номера:

```
1.10 C DEMO
1.20 A X
1.30 S Y=X^2
1.40 P Y
1.50 E
```

Номер состоит из целой и дробной части:

```
1.10
1.20
2.10
2.20
```

Точка в номере не обязана означать группу. Это часть адреса строки и удобный способ оставлять место между соседними строками.

При этом **DO** может использовать целую часть номера как группу. Например, **D 2** вызывает группу строк **2.xx**, а **D 2.10** вызывает одну точную строку **2.10**. Группа появляется только там, где программа явно использует такую форму вызова.

Рекомендуемая сортировка строк - числовая по целой и дробной части номера. Поэтому **1.9** и **1.90** должны считаться одной и той же строкой только если редактор нормализует номера. Чтобы избежать двусмысленности, редактору лучше показывать номер в фиксированной форме:

```
1.10
1.20
1.30
```

Редактор программы

Редактор FOCAL должен быть построчным. Он показывает окно листинга вокруг текущей строки:

- на LCD1602 и других двухстрочных дисплеях - 2 строки программы;
- на графическом дисплее - 4 строки программы.

Текущая строка отмечается маркером > или инверсией:

```
1.10 C DEMO
>1.20 A X
1.30 S Y=X^2
1.40 P Y
```

На двухстрочном дисплее показываются текущая строка и соседняя строка:

```
>1.20 A X
1.30 S Y=X^2
```

Если строка длиннее ширины дисплея, строка прокручивается горизонтально вокруг текстового курсора. Номер строки желательно оставлять видимым, пока это возможно, потому что без номера трудно понять место редактирования.

Навигация в редакторе

Стрелки влево и вправо двигают текстовый курсор внутри текущей строки.

ШГ<- и **ШГ->** двигают текущую строку:

Действие	Клавиша
Курсор на символ влево	стрелка влево
Курсор на символ вправо	стрелка вправо
Предыдущая строка программы	ШГ<-
Следующая строка программы	ШГ->
Принять строку и перейти к следующей	OK / Enter
Сохранить программу и выйти	MENU/ESC
Удалить символ слева	клавиша выбора градусов
Очистить текущую строку	Cx
Удалить пустую текущую строку	Cx на пустой строке

Стрелки не должны случайно перепрыгивать на соседнюю строку при достижении края. Переход между строками выполняется явно через **ШГ<-** и **ШГ->**. Так меньше ошибок на двухстрочном дисплее.

При переходе на последнюю строку и нажатии **ШГ->** редактор может предложить новую строку с автоматически выбранным номером, например текущий номер плюс **0.10**. Номер можно отредактировать вручную.

Ввод операторов

Когда курсор стоит в поле оператора, назначенные клавиши вставляют короткий оператор:

Что нужно вставить	Клавиша
A	a
B	b
C	c
D	d
E	e
F	x
G	Г
P	P
S	x->П
R	В/О

Поле оператора - это место после номера строки и пробела. В других местах те же клавиши работают по контексту выражения или как обычный ввод символов.

Клавиша **x** над **e** вставляет **F** только в поле оператора. В выражениях она остается обычным умножением *****. Буква оператора **F** не вводится клавишей **F**, потому что **F** уже используется как префикс буквенной раскладки.

Клавиша **Г** в поле оператора вставляет латинское **G**. Это клавишная привязка, а не русское ключевое слово: в тексте программы остается оператор **G / GOTO**.

Для **SET** основной сценарий такой:

```
1.10 -> x->П -> 1.10 S
1.10 S X -> x->П -> 1.10 S X=
```

То есть **x->П** в поле оператора вставляет **S**, а внутри левой части **SET** может вставлять знак **=**.

Ввод символов

FOCAL использует тот же принцип префиксов, что и другие текстовые редакторы устройства:

- **F** включает ввод латинской буквы на одно следующее нажатие;
- **K** включает ввод дополнительного символа на одно следующее нажатие;
- после ввода одного символа префикс сбрасывается.

Основные символы:

Символ	Как нажать
пробел	ПП
цифры	цифровые клавиши
.	клавиша десятичной точки
+, -, *, /	соответствующие арифметические клавиши
=	х->П в SET или К ШГ->
(	К ←
)	К →
"	К х<->Y
,	К ПП
;	К ОК / К Enter
!	К С/П
^	К ВП
<	К ШГ<-
>	К -

Двоеточие : в FOCAL первой версии не нужно. В обычной строке размещается один оператор. Точка с запятой ; используется только внутри **FOR**: она отделяет заголовок цикла от единственного оператора тела.

Скобки вводятся через направление: **К ←** - левая скобка, **К →** - правая скобка. Префикс **F** для скобок не используется, потому что он зарезервирован под ввод латинских букв.

Латинские буквы для переменных, функций, комментариев и строк вводятся через **F**. При вводе в позиции, где ожидается оператор, клавиши **a, b, c, d, e, Г, Р, х->П** и **В/О** сначала считаются операторными клавишами. После оператора они могут вводить переменные и текст по обычным правилам.

Локализация

Текущий язык интерфейса влияет на меню, подсказки, сообщения редактора, ввод числа и ошибки. В русском режиме пользователь должен видеть русские сообщения.

Исходный текст программы при этом остается латинским и стабильным:

```
1.10 A X
1.20 S Y=SIN(X)
1.30 P Y
1.40 E
```

Не должно быть русских вариантов операторов или функций. Например, в русском режиме клавиша **a** все равно вставляет **A**, **Г** вставляет **G**, **х->П** вставляет **S**, функция синуса записывается как **SIN(X)**, а случайное число - как **RND()**.

Комментарии и строковые литералы можно показывать по возможностям дисплея и шрифта, но синтаксические части программы остаются латинскими.

Условная компиляция

FOCAL должен отключаться на этапе сборки. Флаг:

```
#ifndef MK61_ENABLE_FOCAL
#define MK61_ENABLE_FOCAL 1
#endif
```

Значение по умолчанию - **1**, то есть FOCAL включен.

Если **MK61_ENABLE_FOCAL** равен **0**, из прошивки должны исключаться:

- пункт меню FOCAL;
- редактор FOCAL;
- парсер и компилятор FOCAL;
- runtime FOCAL;
- хранилище программ FOCAL;
- тестовые или сервисные входы, которые доступны только для FOCAL.

Публичные функции FOCAL в заголовке должны иметь пустые inline-заглушки, чтобы остальной код мог собираться без дополнительных **#ifdef** в местах вызова.

Хранение программ

FOCAL-программа не обязана жить только в оперативной памяти. Рабочая копия нужна в RAM для редактора, парсера и выполнения, но сохраненная программа должна лежать в постоянном хранилище устройства.

Практичная схема:

- при открытии редактора программа загружается из слота в RAM;
- редактор работает с RAM-копией;
- при выходе из редактора или явном сохранении программа записывается обратно в слот;
- при запуске сохраненная программа читается из слота и компилируется или разбирается в рабочее представление;
- если места не хватает, пользователь получает ошибку **FULL?** / **нет места**.

В качестве постоянного хранилища можно использовать существующий слой слотов устройства: SPI flash, а при ее отсутствии - доступное EEPROM-хранилище. Формат слота FOCAL должен отличаться от формата обычной программы МК-61, чтобы режимы не пытались загрузить чужие данные.

Минимальная структура слота:

```
magic/version
type = FOCAL
name
source_length
source_text
checksum
```

Исходником считается канонический текст в короткой форме:

```
1.10 C DEMO
1.20 A X
1.30 S Y=X^2
1.40 P Y,!
1.50 E
```

Компилированное или токенизированное представление можно держать в RAM как кэш, но источником истины должен оставаться сохраненный текст. Так программу проще редактировать, переносить через терминал и обновлять между версиями прошивки.

Язык не должен задавать искусственный лимит на количество строк или длину программы. Ограничение определяется выбранным хранилищем и доступной RAM. При реализации нужны проверки переполнения, но в инструкции это должно формулироваться как "программа не помещается", а не как часть синтаксиса языка.

Операторы

Канонические операторы:

Клавиша	Оператор	Назначение
a	A / ASK	ввод числа
b	B / BRANCH	ветвление по знаку выражения
c	C / COMMENT	комментарий
d	D / DO	выполнить строку или группу как подпрограмму
e	E / EXIT	остановить программу
x	F / FOR	цикл
Г	G / GOTO	безусловный переход
P	P / PRINT	печать значений или текста
x->П	S / SET	присваивание
B/O	R / RETURN	возврат из DO

В листинге можно показывать короткую форму. Полные слова полезны для терминала, импорта и документации, но редактору удобнее работать с короткой формой.

ASK

ASK запрашивает число с клавиатуры и записывает его в переменную:

```
1.10 A X
1.20 A Y
1.30 P X+Y
1.40 E
```

Короткая форма:

```
A X
```

Полная форма:

```
ASK X
```

SET

SET присваивает переменной значение выражения:

```
1.10 S X=2+3
1.20 S Y=SIN(X)
1.30 P Y
1.40 E
```

Короткая форма:

```
S X=2+3
```

Полная форма:

```
SET X=2+3
```

Редактор вводит **SET** клавишей **x->П**, потому что это калькуляторная операция записи значения. В листинге используется явное отображение:

```
S X=2+3
```

Вариант без **S** в листинге не используется. Он хуже читается рядом с другими операторами и усложняет редактор: одна строка FOCAL должна явно начинаться с оператора.

PRINT

PRINT печатает список элементов. Элементом может быть выражение, строковый литерал или управляющий символ !:

```
1.10 P "HELLO"
1.20 P 2+3
1.30 P 2+3, !
1.40 E
```

Короткая форма:

```
P X, !
```

Полная форма:

```
PRINT X, !
```

Запятая разделяет элементы списка. ! завершает текущую строку вывода и переходит к следующей. На маленьком дисплее это можно трактовать как переход к следующей строке экранного вывода или как ожидание перед показом следующего значения.

Примеры:

```
P X
P X, !
P "X=", X, !
```

Исторический FOCAL писал это через **TYPE**, например **TYPE X,!**. В нашей версии это тот же вывод, но записывается только как **PRINT X,!** или коротко **P X,!**.

COMMENT

Комментарий занимает всю строку:

```
1.10 C AREA OF CIRCLE
1.20 A R
1.30 P PI*R^2
1.40 E
```

Текст комментария не выполняется.

EXIT

EXIT завершает выполнение программы:

```
1.10 P "DONE"
1.20 E
```

Историческое имя **QUIT** не используется.

DO и RETURN

DO вызывает подпрограмму. Это не обычный переход: после выполнения указанной строки или группы управление возвращается к строке после **DO**.

Есть две формы:

Форма	Что выполняется	Возврат
D 7.24	только строка 7.24	автоматически после этой строки
D 8	группа 8.xx с первой строки группы	автоматически в конце группы или раньше по RETURN

Вызов точной строки:

```
1.10 A R
1.20 D 2.10
1.30 P S
1.40 E
```

```
2.10 S S=PI*R^2
```

D 2.10 выполняет только строку **2.10**, затем управление возвращается на строку после **DO**.

Вызов группы:

```
1.10 S S=0
1.20 D 2
1.30 P S
1.40 E
```

```
2.10 S S=S+1
2.20 S S=S+2
```

D 2 начинает выполнение с первой существующей строки группы **2.xx** и продолжает до конца этой группы. Конец группы - следующая строка с другой целой частью номера или конец программы.

RETURN нужен только для раннего выхода из группы:

```
2.10 S S=PI*R^2
2.20 B (S-100) 2.30,2.40,2.40
2.30 R
2.40 P S
```


В конце группы **RETURN** писать не обязательно.

RETURN вводится клавишей **B/O**:

```
R  
RETURN
```

Если **RETURN** выполнен вне активного **DO**, это ошибка выполнения.

FOR

FOR повторяет один оператор. Заголовок цикла и тело разделяются точкой с запятой:

```
F I=1,10; P I,!
```

Полная форма:

```
FOR I=1,10; PRINT I,!
```

Диапазон с двумя выражениями:

```
F I=1,10; P I,!
```

означает **I** от **1** до **10** с шагом **1**.

Диапазон с тремя выражениями:

```
F I=1,2,9; P I,!
```

означает **I** от **1** до **9** с шагом **2**. Порядок аргументов выбран в духе FOCAL: начало, шаг, конец.

Исторические примеры:

```
01.01 FOR X=1,10; TYPE X,!  
01.02 FOR X=0,10,100; DO 2
```

в нашей записи становятся:

```
1.01 F X=1,10; P X,!  
1.02 F X=0,10,100; D 2
```

Первая строка однозначна. Вторая строка вызывает группу **2.xx** на каждом шаге цикла.

Если нужно повторять несколько строк, тело цикла должно быть вынесено в подпрограмму:

```
1.10 S S=0  
1.20 F I=1,10; D 2  
1.30 P S  
1.40 E  
  
2.10 S S=S+I
```

Отдельный **NEXT** в первой версии не нужен. Это снижает число операторов и оставляет строку FOCAL самостоятельной: **FOR** либо выполняет простой оператор, либо вызывает подпрограмму через **DO**.

Клавиша ввода **FOR** - * над **e** в поле оператора. Внутри выражения та же клавиша вводит умножение *.

GOTO

GOTO выполняет безусловный переход на указанную строку:

1.10 G 2.10

Короткая форма:

G 2.10

Полная форма:

GOTO 2.10

Адрес должен указывать на существующую строку программы. Если строка не найдена, это ошибка **LINE?**.

Для первой версии лучше требовать точный адрес строки. Сокращение **G 2** можно рассмотреть отдельно: если оно будет поддержано, оно должно означать переход на первую строку группы **2.xx**, а не вызов группы как подпрограммы.

Клавиша ввода **GOTO** - **Г** в поле оператора. В текст программы при этом вставляется латинское **G** .

BRANCH

BRANCH выполняет FOCAL-ветвление по знаку выражения. Безусловный переход делает отдельный оператор **GOTO**.

FOCAL-ветвление по знаку выражения:

B (X) 1.30,1.40,1.50

Смысл трех адресов:

Результат выражения	Переход
меньше нуля	первый адрес
равно нулю	второй адрес
больше нуля	третий адрес

Пример:

1.10 A X
1.20 B (X) 1.50,1.40,1.30
1.30 P X
1.35 E
1.40 P 0
1.45 E
1.50 P -X
1.60 E

Короткая форма:

B (X) 1.50,1.40,1.30

Полная форма:

BRANCH (X) 1.50,1.40,1.30

Отдельный оператор **IF** в первой версии не нужен. Если на клавиатуре хочется подписать клавишу как **IF**, внутри языка это все равно должен быть **BRANCH**.

Переменные

Минимальный набор переменных:

A B C ... Z

Переменные числовые. Строковые переменные в первой версии не нужны.

Позднее можно добавить имена вида:

A0 B7 X1

Но для первой версии лучше остаться на **A - Z**: это проще для редактора, дисплея и ошибок.

Выражения

Поддерживаются:

- числа с десятичной точкой;
- переменные;
- **PI**;
- круглые скобки;
- арифметика **+**, **-**, *****, **/**, **^**;
- унарный минус;
- вызовы встроенных функций.

Примеры:

X+Y
2*PI*R
SIN(A)^2+COS(A)^2
1/(A+B)
SQRT(A^2+B^2)

Возведение в степень пишется как:

X^Y

Оператор ****** в FOCAL не нужен.

Функции МК-61

FOCAL должен использовать калькуляторную семантику МК-61 там, где это возможно. Это особенно важно для тригонометрии, округления, целой/дробной части и **RND()**.

Канонический набор первой версии:

SIN(X)	COS(X)	TG(X)
ASIN(X)	ACOS(X)	ATG(X)
LN(X)	LG(X)	EXP(X)

SQRT(X) ABS(X)
INT(X) FRAC(X) ROUND(X)
SGN(X) MAX(X,Y)

RND()
PI

Алиасы не поддерживаются:

- нет **TAN**, только **TG**;
- нет **ATAN**, только **ATG**;
- нет **LOG**, только **LG**;
- нет **SQR**, вместо этого выражение **X^2**;
- нет **INV**, вместо этого выражение **1/X**;
- нет **EXP10**, вместо этого выражение **10^X**;
- нет **SEED**.

Целая часть, дробная часть и округление

INT(X) соответствует калькуляторной операции **[x]**.

FRAC(X) соответствует калькуляторной операции **{x}**.

ROUND(X) - округление до ближайшего целого. Имя **RND** для округления не используется, потому что **RND()** означает случайное число.

Семантику отрицательных значений нужно проверить по МК-61 и зафиксировать в тестах. Документация FOCAL должна описывать именно поведение калькулятора, а не поведение функций C/C++.

Случайные числа

RND() возвращает результат калькуляторной операции **СЧ**.

1.10 S X=RND()
1.20 P X
1.30 E

Для кубика:

1.10 P 1+INT(6*RND())
1.20 E

SEED не добавляется, потому что такой операции нет в МК-61. Начальное состояние и последовательность должны оставаться такими же, как у калькуляторного генератора.

Тригонометрический режим

Тригонометрические функции должны работать в текущем режиме МК-61:

- градусы;
- радианы;
- грады.

Нужно решить, добавлять ли операторы режима в FOCAL. Возможный вариант:

DEG
RAD
GRAD

Но для первой версии можно считать режим внешним состоянием калькулятора и не добавлять эти операторы в язык. Это решение нужно принять перед реализацией.

Редакторские макросы выражений

Клавиши МК-61 должны не обязательно вставлять имя функции. Некоторые клавиши лучше редактируют уже введенное выражение.

x^2:

X -> X^2
A+B -> (A+B)^2
SIN(X) -> SIN(X)^2

1/x:

X -> 1/X
A+B -> 1/(A+B)
SIN(X) -> 1/SIN(X)

sqrt:

X -> SQRT(X)
A+B -> SQRT(A+B)

abs:

X -> ABS(X)
A-B -> ABS(A-B)

sin, cos, tg, ln, lg, exp:

X -> SIN(X)
A+B -> SIN(A+B)

10^x:

X -> 10^X
A+B -> 10^(A+B)

Правило для редактора:

скобки;

курсором.

- если слева простой атом, операция применяется к нему;
- если слева составное выражение с низким приоритетом, редактор добавляет
- если курсор стоит внутри строки, операция применяется к левому операнду перед

Это позволяет сохранить чистый язык и одновременно оставить калькуляторный ввод.

Примеры программ

Сумма двух чисел

```
1.10 A A
1.20 A B
1.30 P A+B
1.40 E
```

Гипотенуза

```
1.10 C HYPOTENUSE
1.20 A A
1.30 A B
1.40 S C=SQRT(A^2+B^2)
1.50 P C
1.60 E
```

Площадь круга

```
1.10 C CIRCLE AREA
1.20 A R
1.30 S S=PI*R^2
1.40 P S
1.50 E
```

Безусловный переход

```
1.10 G 1.30
1.20 P "SKIP"
1.30 P "OK"
1.40 E
```

Модуль числа через ветвление

```
1.10 A X
1.20 B (X) 1.50,1.40,1.30
1.30 P X
1.35 E
1.40 P 0
1.45 E
1.50 P -X
1.60 E
```

Модуль числа через ABS

```
1.10 A X
1.20 P ABS(X)
1.30 E
```

Подпрограмма

```
1.10 A R
1.20 D 2
1.30 P S
1.40 E

2.10 S S=PI*R^2
```

Цикл FOR

```
1.10 F I=1,5; P I,!
1.20 E
```

Сумма через FOR и DO

```
1.10 S S=0
1.20 F I=1,10; D 2
1.30 P S
1.40 E
```

```
2.10 S S=S+I
```

Случайное число

```
1.10 P RND()
1.20 E
```

Кубик

```
1.10 P 1+INT(6*RND())
1.20 E
```

Ошибки

Ошибки должны локализоваться по текущему языку интерфейса. Внутри реализации можно хранить короткий код ошибки, но на дисплей в русском режиме нужно выводить русский текст.

Минимальные ошибки редактора и исполнителя:

Код	Русский режим	Когда возникает
LINE?	нет строки	строка не найдена
SYNTAX?	синтаксис?	ошибка синтаксиса
VAR?	перем?	неправильное имя переменной
FUNC?	функция?	неизвестная функция
FOR?	цикл?	ошибка в FOR
FULL?	нет места	программа не помещается в RAM или хранилище
RETURN?	нет DO	RETURN без активного DO
STACK?	стек DO	переполнение стека DO
MATH?	матем?	математическая ошибка

Точные тексты можно изменить под ограничения дисплея, но русская локализация для этих ситуаций нужна.

Что не входит в первую версию

- строковые переменные;
- **WHILE**;
- блочный **FOR / NEXT**;
- **GOSUB**, отдельный от **DO**;
- **IF**, отдельный от **BRANCH**;

- алиасы функций;
- русские ключевые слова и русские имена функций;
- **SEED**;
- пользовательские функции;
- произвольные несколько операторов в одной строке вне формы **FOR ...; ...**;
- импорт исторического FOCAL в полном объеме.

Открытые решения перед реализацией

1. Нужно ли добавлять операторы **DEG**, **RAD**, **GRAD**, или оставить режим внешним.
2. Нужно ли поддерживать сокращение **G 2**, и если да, означает ли оно переход на первую строку группы **2.xx**.
3. Каким должен быть точный формат слота FOCAL и как он показывается в общем списке сохраненных программ.
4. Нужен ли позднее блочный **FOR / NEXT**, или достаточно **FOR ...; D 2**.
5. Какие короткие сообщения об ошибках лучше помещаются на дисплей.